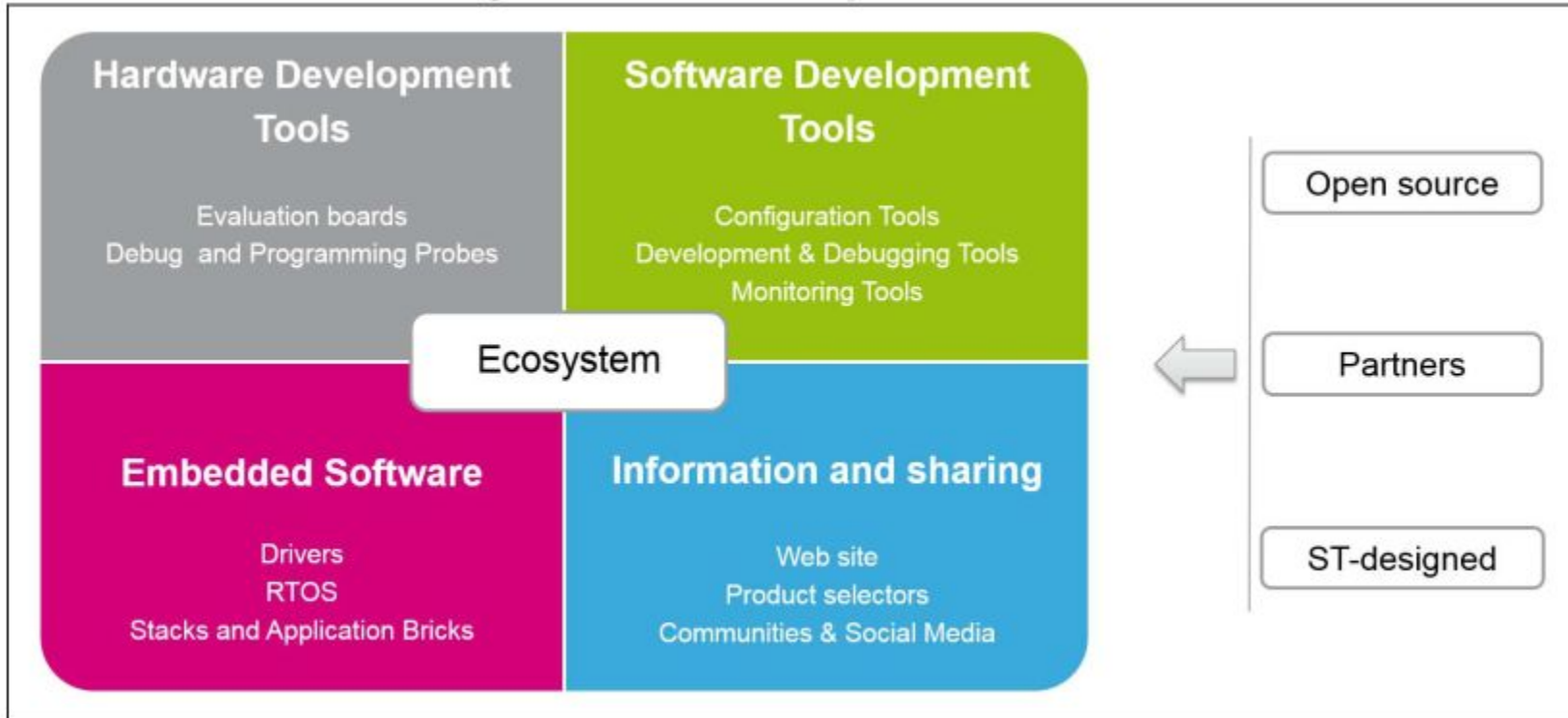


PROGRAMACIÓN DE MICROCONTROLADORES

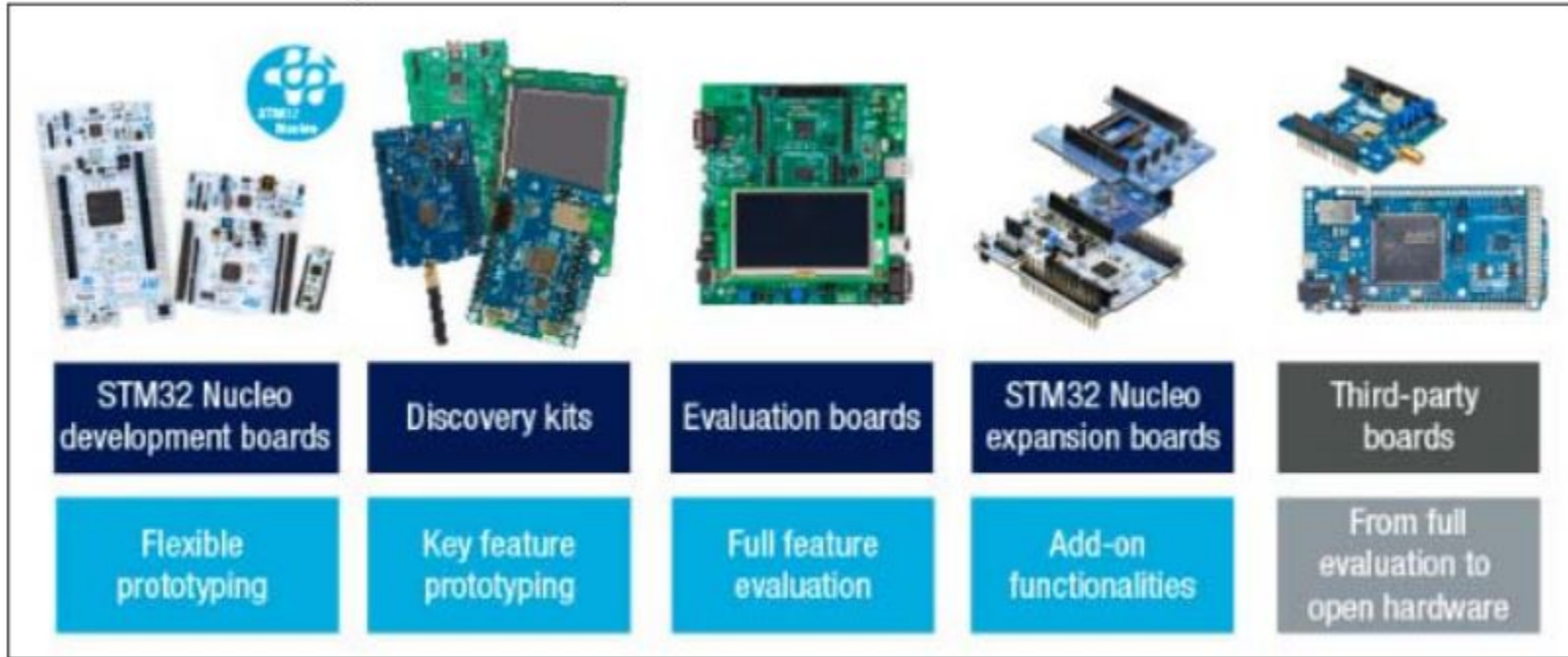
Herramientas de hardware y software

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

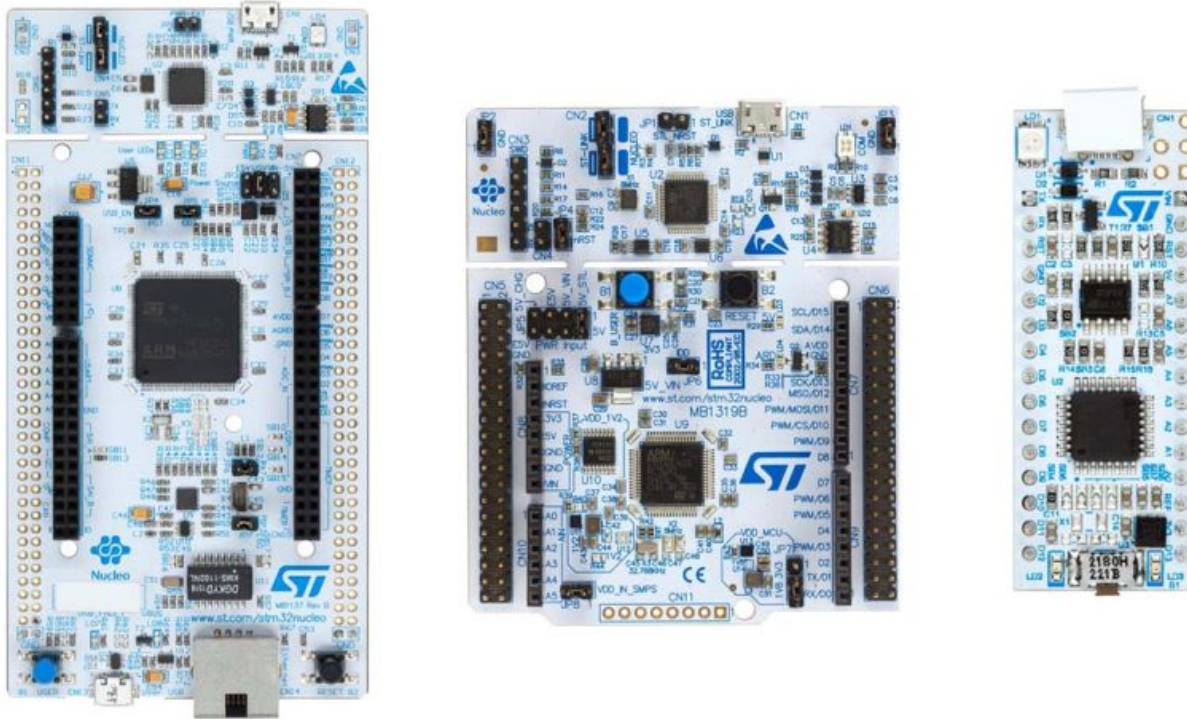
Ecosistema de ST Semiconductors



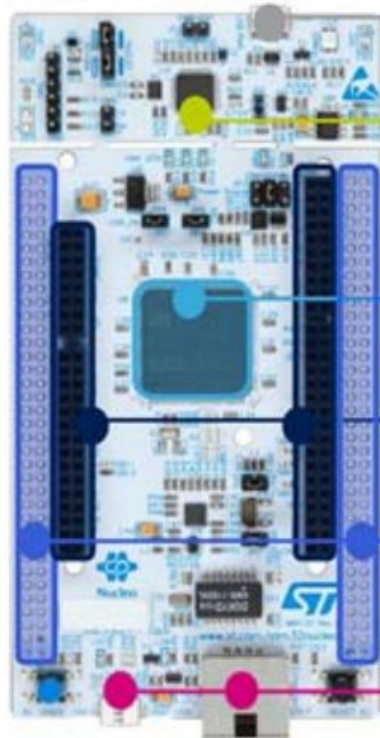
Ecosistema de ST Semiconductors



Ecosistema de ST Semiconductors



Kit de desarrollo NUCLEO-STM32F429ZI



Flexible board power supply:
through USB or external source

Integrated STLink v2-1:
drag & drop device Flash programming

One STM32 MCU flavor with 144 pins

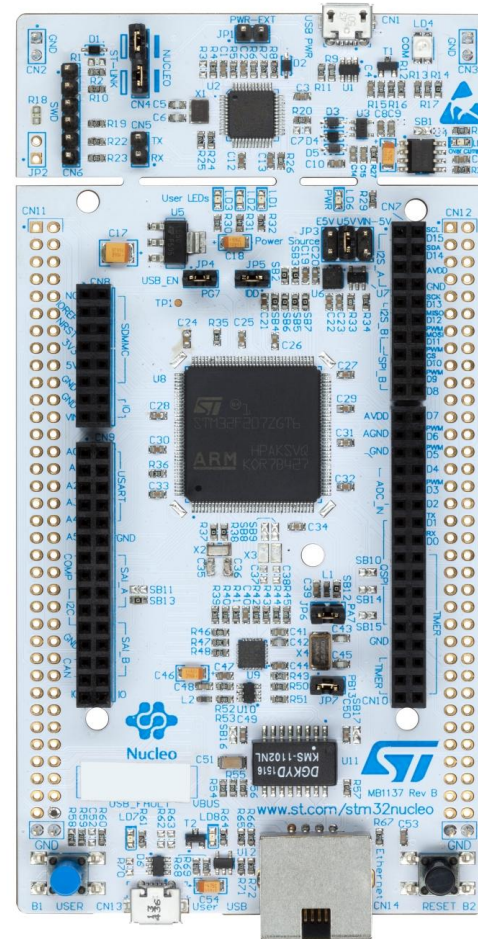
Arduino Uno & ST Zio connectors:
easy access to add-ons

ST morpho extension pins:
direct access to all MCU I/Os

Ethernet & USB OTG (optional)

Plataforma de hardware

- NUCLEO-F429ZI
- STM32F429ZI MCU
- On board ST-LINK debugger/programmer
- STM32Cube MCU Package
- ST Zio connector
- ST morpho headers



Recursos útiles NUCLEO-F446RE

- <https://www.st.com/en/microcontrollers-microprocessors/stm32f446re.html>
- [Datasheet](#) - Product specification
- [Getting started with STM32 Nucleo board software development tools](#)
- [STM32Cube MCU Package for STM32F4 series](#)

MCU STM32F446RE

- Arm® Cortex®-M4 32-bit RISC core @180Mhz
- Floating point unit (FPU)
- Full set of DSP instructions
- 512 Kbytes of Flash memory/128+4 KB RAM
- USB OTG HS/FS, 17 TIMs, 3 ADCs and 20 communication interfaces
- MCU STM32f446RE [manual](#)

Recursos útiles para STM32F4

- <https://www.st.com/en/microcontrollers-microprocessors/stm32f429zi.html>
- [Datasheet](#)
- [Reference Manual](#)
- [Programming Manual](#)

- [Cortex-M4 Technical Reference Manual](#)

Plataforma de software - STM32CubeIDE

- Eclipse®/CDT framework
- GCC toolchain para desarrollo y GDB para debugging
- STM32CubeMX (asistente gráfico para generar código de inicialización)
- Soporte para STM32 MCUs y boards (generación automática de proyectos)
- <https://www.st.com/en/development-tools/stm32cubeide.html>

STM32CubeIDE

The screenshot displays the STM32CubeIDE interface with the following components:

- Project Explorer:** Shows the project structure for 'Ej0_blinky', including 'Binaries', 'Includes', 'Drivers', 'Inc', and 'Src'. The 'main.c' file is selected under 'Src'.
- main.c:** The main program file is open, showing the following code:

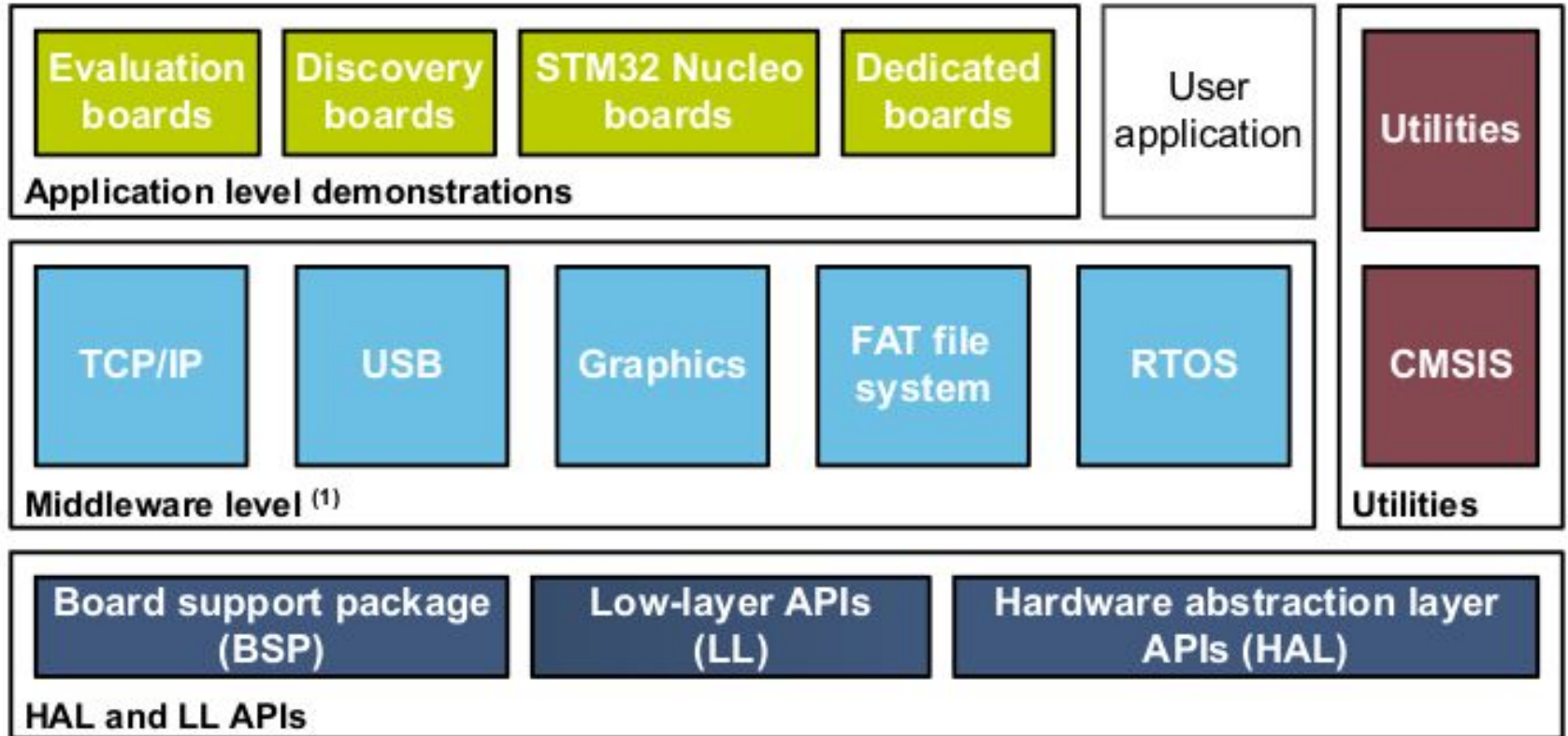
```
469/**
47  * @brief Main program
48  * @param None
49  * @retval None
50  */
51 int main(void)
52 {
53     /* STM32F4xx HAL library initialization:
54      * - Configure the Flash prefetch
55      * - SysTick timer is configured by default as source of time base, but user
56      *   can eventually implement his proper time base source (a general purpose
57      *   timer for example or other time source), keeping in mind that Time base
58      *   duration should be kept 1ms since PPP_TIMEOUT_VALUES are defined and
59      *   handled in milliseconds basis.
60      * - Set NVIC Group Priority to 4
61      * - Low Level Initialization
62     */
63     HAL_Init();
64
65     /* Configure the system clock to 180 MHz */
66     SystemClock_Config();
67
68     /* Initialize BSP Led for LED2 */
69     BSP_LED_Init(LED2);
70
71     /* Infinite loop */
72     while (1)
73     {
74         BSP_LED_Toggle(LED2);
75         HAL_Delay(100);
76     }
77 }
78
79
80 /**
81  * @brief System Clock Configuration
82  * The system clock is configured as follow :
83  * System Clock source = PLL (HSE)
```
- Outline:** Shows the function declarations in 'main.h':
 - main.h
 - UartHandle: UART_HandleTypeDef
 - SystemClock_Config(void): void
 - Error_Handler(void): void
 - main(void): int
 - SystemClock_Config(void): void
 - Error_Handler(void): void
 - assert_failed(uint8_t, uint32_t): void
- Build Analyzer:** Shows the build results for 'Ej0_blinky.elf' on 22 Jun. 2023 at 15:20:24. It includes a table of function complexity analysis:

Function	Location	Complexity
HAL_RCC_OscConfig	stm32f4xx_hal_rcc.c:221	61
HAL_GPIO_Init	stm32f4xx_hal_gpio.c:164	22
HAL_RCC_ClockConfig	stm32f4xx_hal_rcc.c:593	20

STM32Cube MCU Package

- STM32Cube hardware abstraction layer (HAL).
- Low-layer (LL) APIs.
- Middleware (RTOS, USB, FAT file system, Graphics and TCP/IP).
- TouchGFX graphic software stack.
- Ejemplos de uso de periféricos y demos.
- HAL y LL APIs: compliance with MISRA-C:2004 guidelines.
- STM32F4 HAL and low-layer drivers: [User Manual](#)

STM32Cube MCU Package



CMSIS

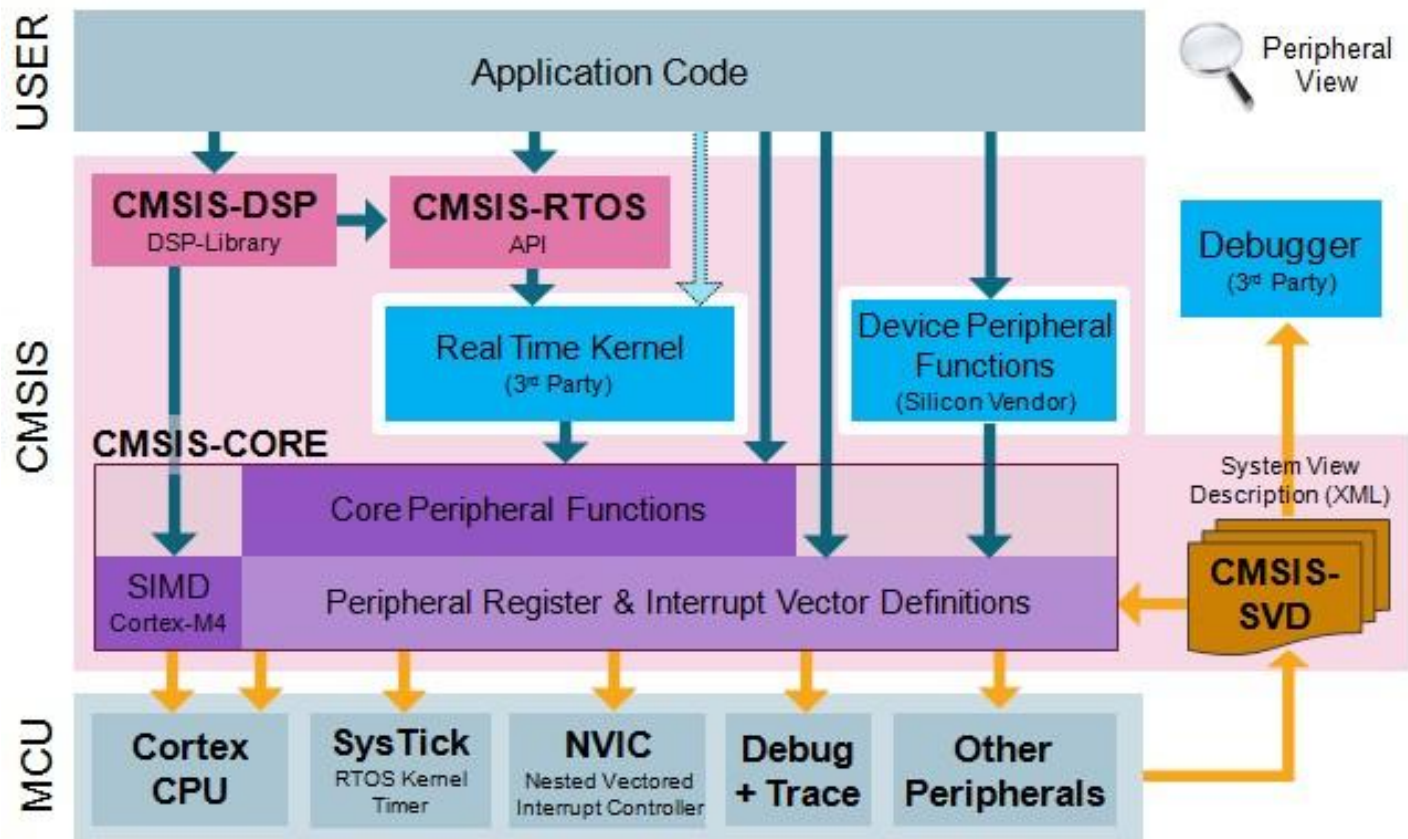
- Estándar de interfaz de software del microcontrolador ARM® Cortex
- HAL independiente del fabricante.



工具供应商：



CMSIS

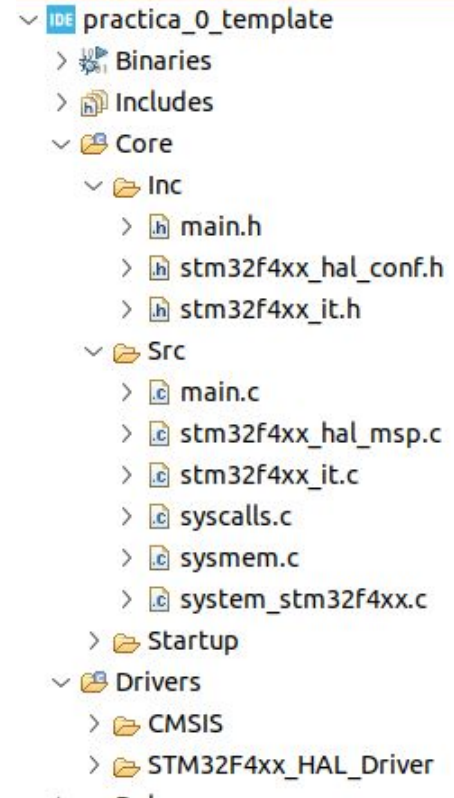


CMSIS

- CMSIS-CORE: proporciona la **interfaz entre los procesadores** Cortex-M0, Cortex-M3, Cortex-M4, SC000 y SC300 **y los registros de periféricos**
- CMSIS-DSP: una biblioteca DSP que contiene más de 60 funciones implementadas en punto fijo (fracción q7, q15, q31) y punto flotante de precisión simple (32 bits)
- API CMSIS-RTOS: una interfaz de programación estandarizada para sistemas operativos en tiempo real para control de subprocesos, gestión de recursos y tiempo
- CMSIS-SVD: un archivo XML de descripción de los periféricos de un dispositivo que se puede usar para “alertar” al debugger sobre su existencia.

Practica_0_template - Estructura de archivos

- Drivers
 - CMSIS
 - HAL
- Core
 - stm32f4xx_hal_conf.h
 - stm32f4xx_it.c/h
 - stm32f4xx_hal_msp.c
 - Startup: SP, PC ISR vector table, llama a main()



Practica_0_template - Código

```
int main(void)
{
    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */

        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
        HAL_Delay(500);

        if (!HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin)){
            HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, 1);
            while(1);
        }

    }
    /* USER CODE END 3 */
}
```